Wistia Video Analytics Pipeline

AWS Serverless Medallion Architecture (Bronze to Gold) Implementation Manual

AWS REGION	ARCHITECTURE	AUTH STANDARD	ORCHESTRATION
us-east-2 (Ohio)	Medallion (Bronze/Gold)	Bearer Token Header	EventBridge Cron + SQS

1. End-to-End Pipeline Architecture

The Wistia Video Analytics batch pipeline ingests video performance metrics from the Wistia REST API, stores raw JSON in a Bronze S3 bucket, queues processing pointers in SQS, and enriches data into structured Gold JSON files ready for analytics.



2. Step 1: Create the Wistia Delay SQS Queue

- Navigate to **Amazon SQS** in `us-east-2` and click **Create queue**.
- Set Type to **Standard** and Name to `wistia-delay-queue`.
- Set Delivery Delay to `10 seconds` (allows Bronze S3 put consistency).
- Copy the generated **Queue URL** for environment variable configuration.

3. Step 2: Configure & Deploy Ingestion Lambda

Function Name: `wistia-ingest-lambda-sd` | Environment Variable: `SQS_QUEUE_URL`

```
import json, os, urllib.request, boto3 s3_client = boto3.client('s3') sqs_client = boto3.cli
ent('sqs') BRONZE_BUCKET = os.environ.get('BRONZE_BUCKET_NAME', 'insightflow-bronze-bucket-sd') SQ
S_QUEUE_URL = os.environ.get('SQS_QUEUE_URL') API_TOKEN = "0323ade64e13f79821bdc0f2a9410d9ec3873aa
9df01f8a4a54d4e0f3dd2e6b4" MEDIA_IDS = ["8hunphufxp", "9k4tbcdfg0"] def fetch_wistia_stats(media_i
d): url = f"https://api.wistia.com/v1/stats/medias/{media_id}.json" req = urllib.request.Request(u
rl) req.add_header('Authorization', f'Bearer {API_TOKEN}') try: with urllib.request.urlopen(req) a
s response: return json.loads(response.read().decode('utf-8')) except Exception as e: print(f"Erro
r fetching Wistia stats for {media_id}: {str(e)}") return None def lambda_handler(event, context):
for media_id in MEDIA_IDS: data = fetch_wistia_stats(media_id) if data: s3_key = f"wistia/wistia_s
tats_{media_id}.json" s3_client.put_object(Bucket=BRONZE_BUCKET, Key=s3_key, Body=json.dumps(dat
a), ContentType='application/json') if SQS_QUEUE_URL: message_body = {"media_id": media_id, "s3_ke
y": s3_key, "bucket": BRONZE_BUCKET} sqs_client.send_message(QueueUrl=SQS_QUEUE_URL, MessageBody=j
son.dumps(message_body)) return {'statusCode': 200, 'body': json.dumps('Wistia batch ingestion com
pleted successfully!')}
```

4. Step 3: Configure & Deploy Enrichment Lambda

Function Name: `wistia-enrich-lambda-sd` | Trigger: **SQS** (`wistia-delay-queue`)

```
      import json, os, boto3 s3_client = boto3.client('s3') GOLD_BUCKET = os.environ.get('GOLD_BUC
KET_NAME', 'insightflow-gold-bucket-sd') def lambda_handler(event, context): records = event.get
('Records', [ {"body": json.dumps(event)} ]) for record in records: body = json.loads(record.get
('body', json.dumps(event))) payload = body if isinstance(body, dict) else json.loads(body) media_
id = payload.get('media_id') if 's3_key' in payload and 'bucket' in payload: obj = s3_client.get_o
bject(Bucket=payload['bucket'], Key=payload['s3_key']) wistia_data = json.loads(obj['Body'].read
().decode('utf-8')) else: wistia_data = payload if not media_id: media_id = wistia_data.get('medi
a', {}).get('hashed_id', 'unknown') # Extract & Map Clean Wistia Engagement Metrics gold_record =
{ "media_id": media_id, "load_count": wistia_data.get('load_count', 0), "play_count": wistia_data.
get('play_count', 0), "unique_visitors": wistia_data.get('visitors', 0), "play_rate": wistia_data.
get('play_rate', 0.0), "hours_watched": wistia_data.get('hours_watched', 0.0), "engagement": wisti
a_data.get('engagement', 0.0), "raw_payload": wistia_data } s3_client.put_object( Bucket=GOLD_BUCK
ET, Key=f"gold-wistia-{media_id}.json", Body=json.dumps(gold_record), ContentType='application/jso
n' ) print(f"Successfully wrote Gold Wistia record for media {media_id}") return {'statusCode': 20
0, 'body': json.dumps('Wistia enrichment completed!')}
```

5. Step 4: Automate via Amazon EventBridge

- Open `wistia-ingest-lambda-sd` in the Lambda Console.
- Click **Add Trigger** and select **EventBridge (CloudWatch Events)**.
- Select **Create a new rule** and name it `WistiaDailyIngestRule`.
- Set Schedule Expression to `rate(1 day)` or `cron(0 6 * * ? *)`.
- Click **Add**. (*Ensure trigger is NOT attached to the enrichment function*).

6. Field Mapping & Gold Schema Verification

> **Validation Confirmation**
> Verified raw API response mapping against targeted Gold metrics for media IDs `8hunphufxp` and `9k4tbcdfg0`.

Wistia API Key	Gold Schema Field	Data Type	Description
`hashed_id`	`media_id`	String	Unique alphanumeric Wistia video identifier
`load_count`	`load_count`	Integer	Total video player embed loads / impressions
`play_count`	`play_count`	Integer	Total times video playback was initialized
`visitors`	`unique_visitors`	Integer	Count of distinct unique IP viewers
`play_rate`	`play_rate`	Float	Ratio of plays to total loads (plays / loads)
`hours_watched`	`hours_watched`	Float	Cumulative duration of content consumed in hours